

POLITECNICO
MILANO 1863

2022

SINTESI EURISTICA DI RETI COMBINATORIE 2 LIVELLI

Reti Logiche

Fabio Salice

Sintesi reti combinatorie a 2 livelli: Metodi euristici

- La minimizzazione esatta presenta due problemi:
  - L'enorme numero di implicanti primi.
    - Può essere dimostrato che il numero degli implicanti primi di una funzione logica di n ingressi può essere maggiore di $3^n/n$.
  - L'intrattabilità del problema di copertura.
    - E' un problema NP-completo.
- Soluzione:
  - Miglioramento iterativo della soluzione: partendo da una **condizione iniziale** (specifiche della funzione) la copertura è modificata per **cancellazione**, **aggiunta** e **modifica** di implicanti fino a che non è raggiunta una condizione di minimalità cioè quando nessuna delle trasformazioni porta a successivi miglioramenti.

Sintesi reti combinatorie a 2 livelli: Metodi euristici

Name	In/Out	Terms	Primes	Ess
accpla	50/69	183	1758057	97
ex4	128/28	620	1.8348e14	138
ibm	48/17	173	1047948800	172
jbp	36/57	166	2496809	0
mainpla	27/54	181	87692	29
misg	56/23	75	6499491839	3
mish	94/43	91	1.1243e15	3
misj	35/14	48	139103	13
pdc	16/40	2406	23231	2
shift	19/16	100	165133	100
signet	39/8	124	78735	104
soar.pla	83/94	529	3.3047e14	2
ti	47/72	241	836287	46
ts10	22/16	128	524280	128
x2dn	82/56	112	1.1488e16	2
x7dn	66/15	622	566698631	378
xparc	41/73	551	15039	140
ex5	8/63	256	2532	28
max1024	10/6	1024	1278	14
prom2	9/21	287	2635	9
ex1010	10/10	810	25888	0
test2	11/35	1999	109099	0
test3	10/35	1003	41344	0

Sintesi reti combinatorie a 2 livelli: Metodi euristici

http://www.problemistics.org/corso/prima.pagina.html
Corso su
Problemistica
L'arte & la tecnica di affrontare problemi

- Metodo:
  - modo di procedere secondo una certa sequenza (tempo) ed un certo scenario organizzativo (spazio) al fine di raggiungere un obiettivo (soluzione) in relazione ad una insoddisfazione (problema).
    - Algoritmico, Euristico o misto
- Algoritmico:
  - metodo di risoluzione di problemi che poggia su un insieme finito di regole ben definite (Algoritmo) e che conduce alla soluzione del problema.
- Algoritmo Euristico:

Regole empiriche: guide generali che assistono/indirizzano nella presa di decisioni su cosa fare/dove andare alla ricerca della soluzione del problema.

  - approccio alla risoluzione di problemi che attiene a metodi che si affidano a regole empiriche generali che poggiano sul buon senso e che si ritiene possano condurre alla soluzione del problema.

Sintesi reti combinatorie a 2 livelli: Metodi euristici

- Minimizzazione del numero di implicanti o letterali
  - Problema NP-completo
    - calcolo degli implicanti primi
    - calcolo della copertura di costo minimo
  - Metodo di Quine-McCluskey: identificazione di tutti gli implicanti primi (espansione) e di copertura (essenzialità e dominanze)
    - Implementato in Espresso Exact
  - Espresso: metodo euristico
    - Binary cube notation
  - Scherzo: metodo euristico
    - basato sui BDD e sui set combinatori,
    - due ordini di grandezza piú veloce di Espresso

Sintesi reti combinatorie a 2 livelli: Metodi euristici

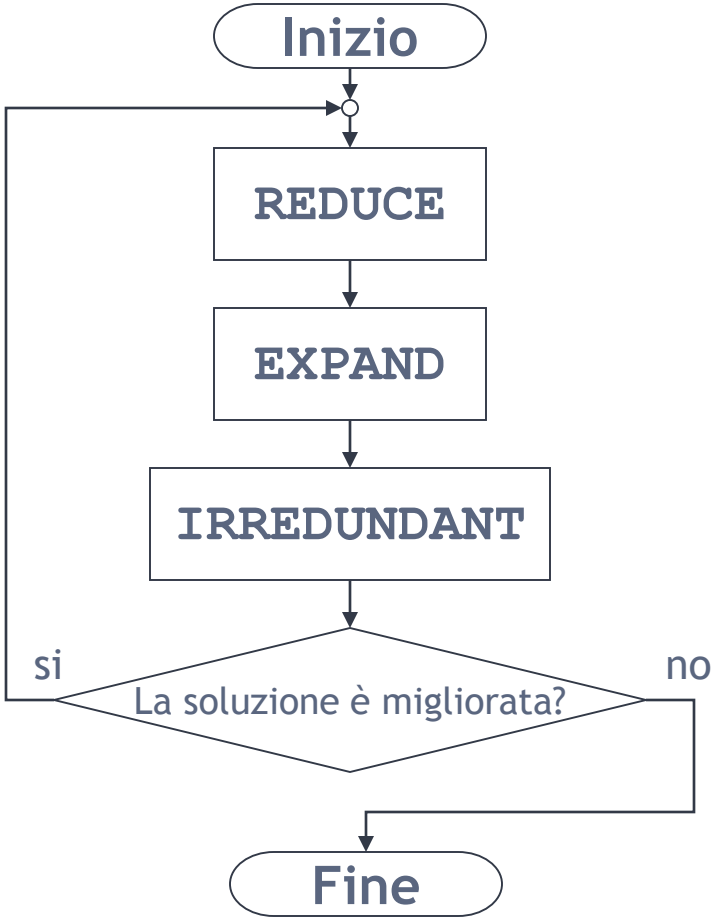
- I metodi euristici di minimizzazione differiscono per qualità della soluzione rispetto a quelli esatti
  - **Qualità**: Differenza in cardinalità tra la copertura minima ottimale (euristica) e quella minima ottima (ottenuta con metodi esatti).
- Le soluzioni prodotte da Espresso (minimizzazione logica euristica a 2 livelli) coincidono, spesso, con quelle di Espresso-Exact, ma in tempi più brevi.
  - Procedura di minimizzazione:
    - Ingresso: ON-set (Lista dei mintermini/implicanti) ed il DC-set della funzione.
    - Condizione iniziale: L'ON-set (la lista degli implicanti) rappresenta la copertura iniziale della funzione.
    - Sviluppo: La copertura iniziale viene iterativamente manipolata da alcuni operatori.
    - Termine: L'operazione si conclude quando nessun operatore migliora la copertura.

Sintesi reti combinatorie a 2 livelli: Metodi euristici: Operatori

- Gli operatori utilizzati da Espresso sono:
  - Expand
    - espande i cubi rendendoli primi. La copertura ottenuta risulta costituita da soli implicanti primi (copertura prima)
      - Nota: eliminazione di uno o più letterali. Il risultato non interseca l'OFFset
  - Reduce
    - riduce i cubi per consentire di uscire da minimi locali dopo l'espansione
    - la copertura ottenuta dopo una riduzione non è più di soli implicanti primi ma è della **stessa cardinalità di quella di partenza**.
      - Non è aumentato il numero degli implicanti
    - Nota: dati due implicanti A e B la riduzione di A è A-B (booleano: A*B'). Reduce richiede che non aumenti la cardinalità
  - Irredundant
    - elimina i cubi ridondanti. Modifica la cardinalità della copertura riducendola.

Sintesi reti combinatorie a 2 livelli: Metodi euristici

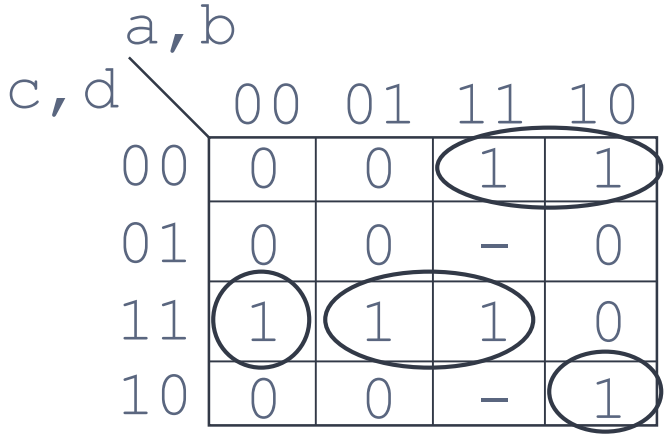
- Algoritmo:



Sintesi reti combinatorie a 2 livelli: Metodi euristici

- Esempio:

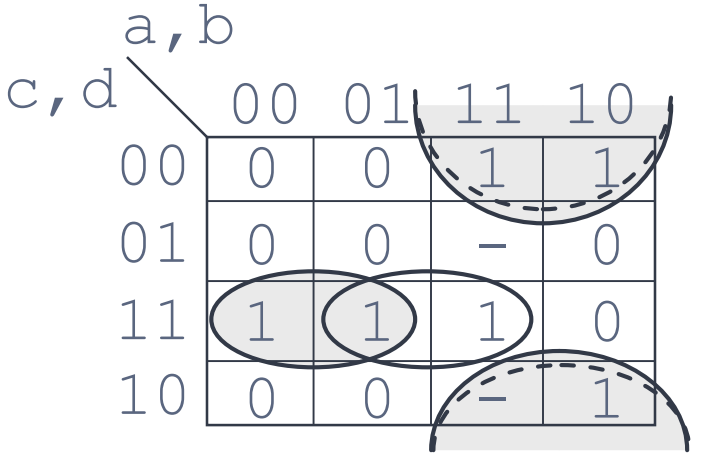
Condizione iniziale:
deriva direttamente dalle specifiche del problema



Cardinalità: 4

Costo in letterali: 14

Espansione



Cardinalità: 3

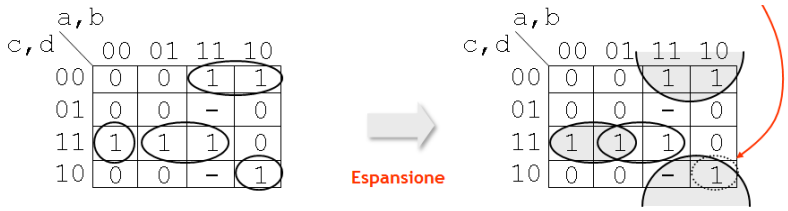
Costo in letterali: 8

Nota: copertura non prima.
Nota: la riduzione non ha effetto perché non ci sono cubi sovrapposti.

Nota: copertura prima. Inoltre, non esistono implicanti ridondanti.

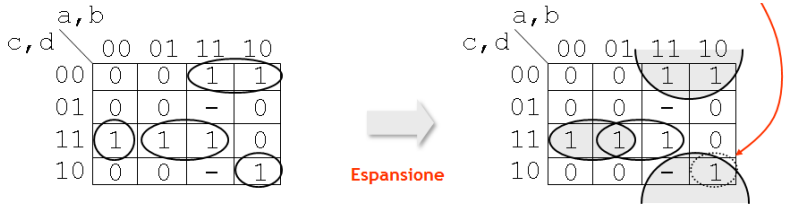
Sintesi reti combinatorie a 2 livelli: Metodi euristici

- Esempio (Cont.):
  - **Processo algebrico di espansione**: l'espansione di un termine prodotto corrisponde, algebricamente, all'eliminazione di un letterale. L'espansione deve essere verificata: è valida solo se non interseca OFFset
    - ONset: {ac'd', a'b'cd, bcd, ab'cd'}
    - OFFset: {a'c', a'd', ab'd} → a'c'+a'd'+ab'd



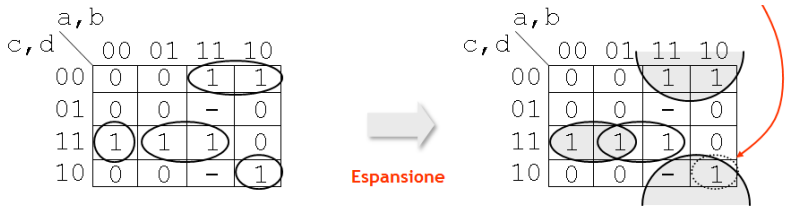
  - Espansione di ac'd'
    - c'd': c'd' * (a'c'+a'd'+ab'd) = a'c'd' + a'c'd' = a'c'd' ≠ ∅ → NON VALIDA
      - Verifica della validità dell'espansione; se interseca l' OFFset non è valida.
    - ad': ad' * (a'c'+a'd'+ab'd) = 0 → VALIDA
      - Espansione di ad'
        - a: a* (a'c'+a'd'+ab'd) = ab'd → NON VALIDA
        - d': d'* (a'c'+a'd'+ab'd) = a'c'd'+a'd'=a'd' → NON VALIDA
    - ac': ac' * (a'c'+a'd'+ab'd) = ab'c'd → NON VALIDA

Sintesi reti combinatorie a 2 livelli: Metodi euristici



- Esempio (Cont.):
  - Espansione di a'b'cd
    - b'cd : b'cd * (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
    - a'cd : a'cd * (a'c'+a'd'+ab'd) = ∅ → VALIDA
      - Espansione di a'cd
        - cd : cd* (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
        - a'd : a'd* (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
        - a'c : a'c* (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
    - a'b'd : a'b'd * (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
    - a'b'c : a'b'c * (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
  - Espansione di bcd
    - bc: c'd' * (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
    - bd: ad' * (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
    - cd: ac' * (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA

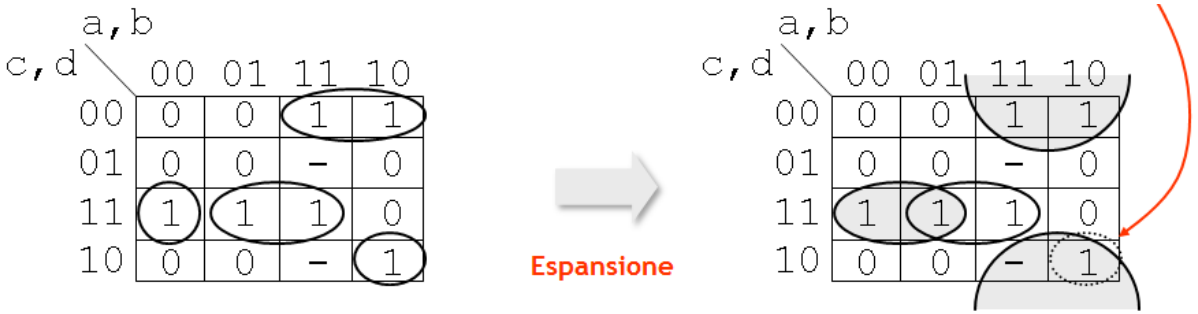
Sintesi reti combinatorie a 2 livelli: Metodi euristici



- Esempio (Cont.):
  - Espansione di ab'cd'
    - b'cd' : b'cd' * (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
    - acd' : a'cd' * (a'c'+a'd'+ab'd) = ∅ → VALIDA
      - Espansione di acd'
        - cd' : cd'* (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
        - ad' : ad'* (a'c'+a'd'+ab'd) = ∅ → VALIDA
        - ac : ac* (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
    - ab'd' : ab'd' * (a'c'+a'd'+ab'd) = ∅ → VALIDA
      - Espansione di ab'd'
        - b'd' : b'd'* (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
        - ab' : ab'* (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA
        - ad' : ad'* (a'c'+a'd'+ab'd) = ∅ → VALIDA
    - ab'c : a'b'c * (a'c'+a'd'+ab'd) ≠ ∅ → NON VALIDA

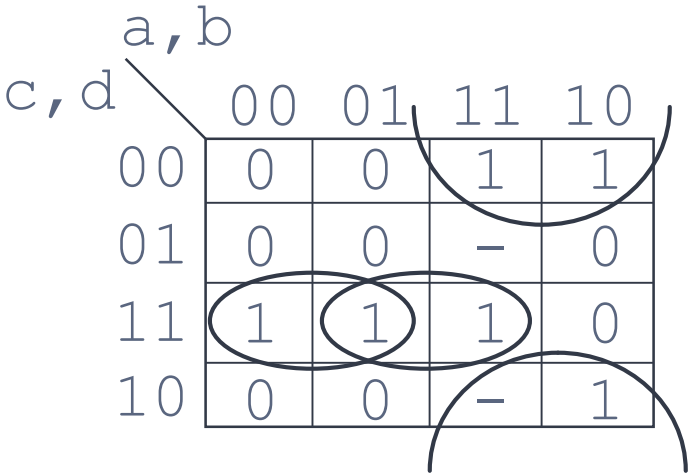
Sintesi reti combinatorie a 2 livelli: Metodi euristici

- Esempio (Cont.): Riassumendo
  - Partendo da ONset: {ac'd', a'b'cd, bcd, ab'cd'} , OFFset: {a'c', a'd', ab'd}
  - Le espansioni valide sono:
    - ac'd' → ad'
    - a'b'cd → a'cd
    - bcd → bcd
    - ab'cd' → ad'
  - Si ottiene ONset: {ad', a'cd, bcd}, OFFset: {a'c', a'd', ab'd}



Sintesi reti combinatorie a 2 livelli: Metodi euristici

- Esempio (cont.): la soluzione è migliorata, quindi si prosegue

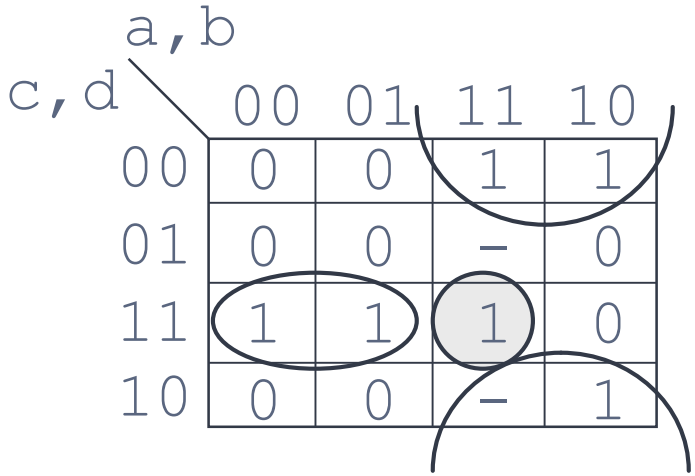


Cardinalità: 3

Costo: 8

Nota: copertura prima

Riduzione



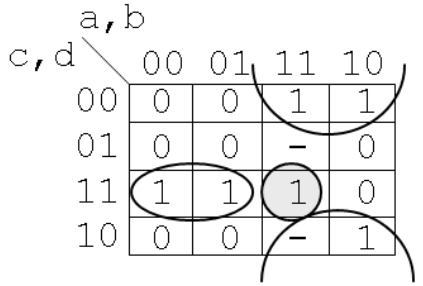
Cardinalità: 3

Costo: 9

Nota: copertura non prima

Sintesi reti combinatorie a 2 livelli: Metodi euristici

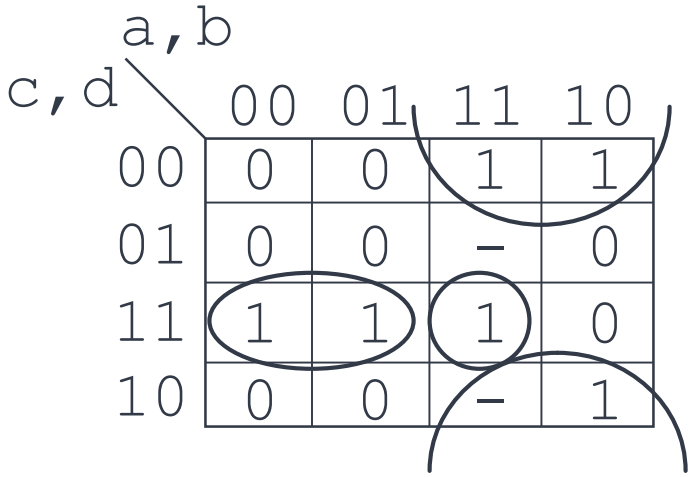
- Esempio (Cont.):
  - **Processo algebrico di riduzione**: la riduzione di un termine prodotto corrisponde, algebricamente, alla sottrazione booleana tra due implicanti. In Reduce una riduzione è valida se solo se la cardinalità dell'insieme degli implicanti non aumenta.
  - Di ONset: {ad', a'cd, bcd}, gli implicanti che hanno delle parti in comune sono:
    - ad' * a'cd = ∅
    - ad' * bcd = ∅
    - a'cd * bcd = a'bcd ← unica coppia su cui applicare la riduzione



  - Riduzione di a'cd rispetto a bcd
    - bcd – a'cd → bcd*(a'cd)' = bcd * (a+c'+d') = abcd
    - Riassumendo: non aumenta la cardinalità dell'insieme degli implicanti ONset: {ad', a'cd, abcd}

Sintesi reti combinatorie a 2 livelli: Metodi euristici

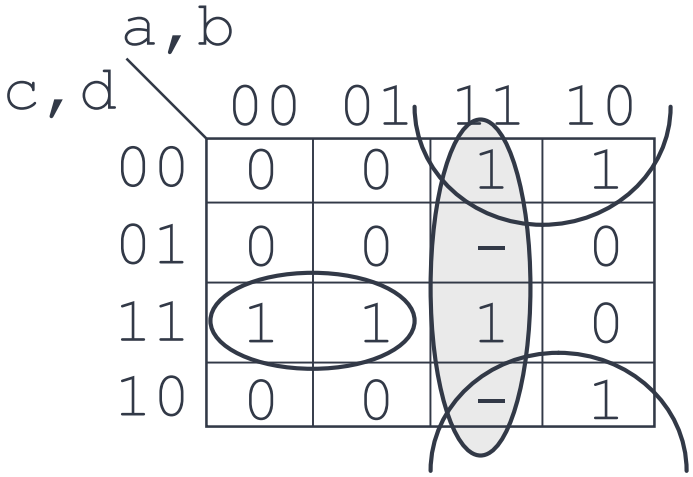
- Esempio (cont.):



Cardinalità: 3
Costo: 9

Nota: copertura non prima

Espansione

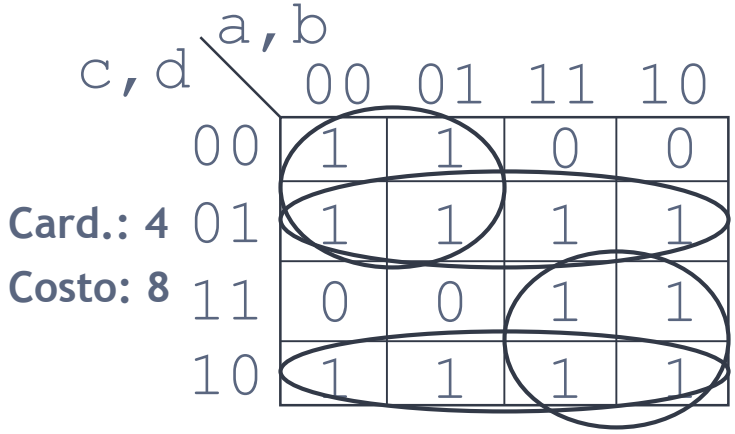


Cardinalità: 3
Costo: 7

Nota: copertura prima
Fine della procedura (l'applicazione di una riduzione porta alla soluzione individuata al passo precedente)

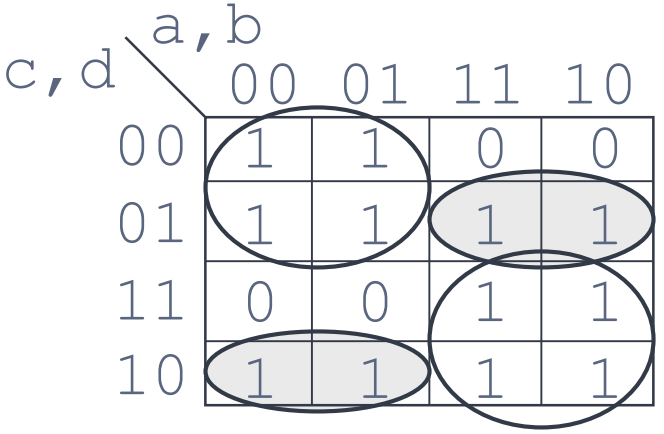
Sintesi reti combinatorie a 2 livelli: Metodi euristici

- Esempio 2:



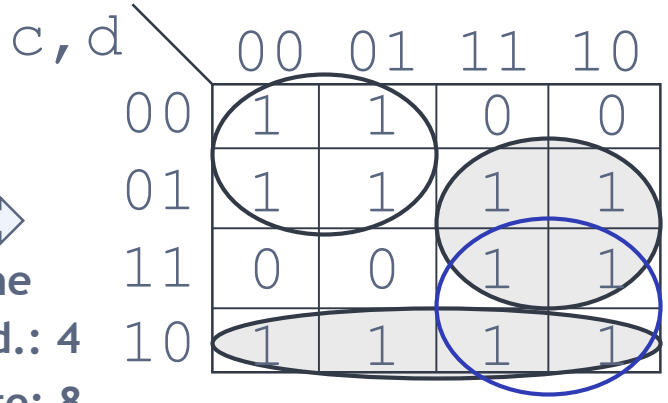
Card.: 4
Costo: 8

Riduzione



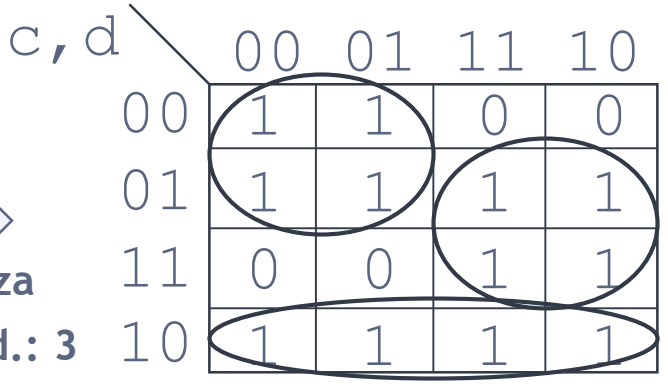
Card.: 4
Costo: 10

Espansione



Card.: 4
Costo: 8

Ridondanza



Card.: 3
Costo: 6

Sintesi reti combinatorie a 2 livelli: Metodi euristici: Expand

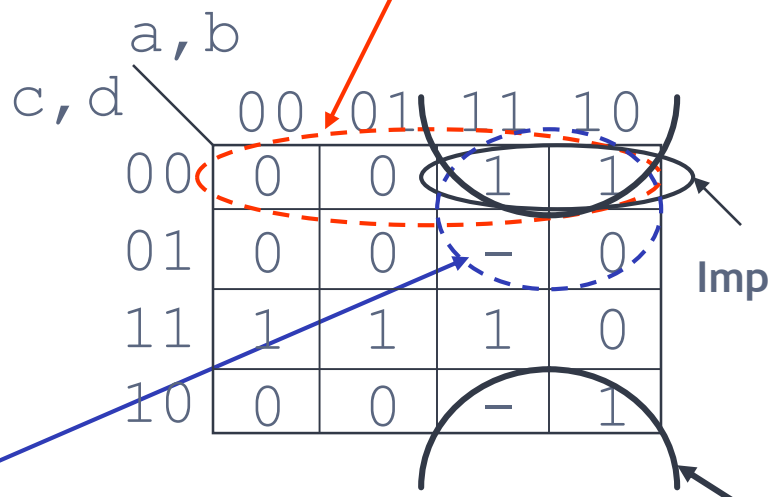
- Expand
  - Gli implicanti della copertura sono rielaborati uno alla volta. Ogni implicante è espanso a primo e tutti gli implicanti da esso completamente coperti sono eliminati.
  - L'operatore Expand rende la copertura prima e minimale. L'espansione di un implicante è realizzata aumentando il sotto cubo ad esso associato in una o più direzioni e verificando se l'espanso è ammissibile
  - Verifica dell'ammissibilità dell'espansione:
    - Una espansione è ammissibile se l'implicante ottenuto non interseca l'OFFset.
      - E' richiesta la conoscenza dell'OFFset e questo può essere pesante in termini di memoria utilizzata.

Sintesi reti combinatorie a 2 livelli: Expand – Verifica Ammissibilità

OFF-Set: a'c'+ab'd+a'd'

Espansione rispetto ad a di ac'd' è c'd'
espansione non ammissibile

Verifica ammissibilità:
OFF-Set*(c'd')= a'c'd'≠ 0
non ammissibile poiché l'intersezione con l'OFF-set non è nullo



Implicante da espandere: a c'd'

Espansione rispetto ad d di ac'd' è ac'
espansione non ammissibile (intersezione = ab'c'd)

Espansione rispetto a c di ac'd' è ad'
espansione ammissibile (intersezione = 0)

APPROFONDIMENTO FACOLTATIVO

Sintesi reti combinatorie a 2 livelli: Expand – Verifica Ammissibilità

- L'espansione può essere effettuata utilizzando la *Positional Cube Notation:* 0 è codificato con 10, 1 è codificato con 01, - è codificato con 11 , 00 non è una codifica ammissibile
  - È esattamente equivalente al processo booleano descritto in precedenza

OFF-Set: a'c'+ab'd+a'd' ⇨

a'c'	[10 11 10 11]
ab'd	[01 10 11 01]
a'd'	[10 11 11 10]

ac'd' = [01 11 10 10]

Espansione di ac'd' rispetto ad a → [11 11 10 10]

[10 11 10 11]
[01 10 11 01] & [11 11 10 10] = [01 10 10 00] → non ammissibile
[10 11 11 10]
[10 11 10 10]

Espansione di ac'd' rispetto ad c' → [01 11 11 10]

[10 11 10 11]
[01 10 11 01] & [01 11 11 10] = [01 10 10 00] → ammissibile
[10 11 11 10]
[00 11 10 10]

Sintesi reti combinatorie a 2 livelli: Expand – Ordine di espansione implicanti

- La qualità del risultato dipende da due fattori:
  - Validità della espansione del singolo implicante. Soluzione: si traduce l'espansione in un problema di copertura.
    - Matrice di blocco: OFFset in relazione con i letterali del cubo da espandere
  - Direzione della espansione. Soluzione: si utilizza una euristica che consente sviluppare l'espansione nella direzione più conveniente
    - Copertura ottima della «matrice di blocco»
  - Ordine di espansione
    - Gli implicanti sono ordinati in base alla probabilità sia di essere espansi che di non essere coperti da altri implicanti. Utile se accompagnata da una eliminazione di ridondanze (o sovrapposizione) permettendo di velocizzare l'espansione
    - Ad ogni implicante è associato un peso che misura la sua propensione alla espansione e alla non-copertura da parte di altri implicanti. L'implicante con **peso minore** è quello che ha più probabilità di essere espanso e non coperto da altri. Per il calcolo del peso si utilizza, per i letterali, la notazione **positional-cube**.

Sintesi reti combinatorie a 2 livelli: Expand – Matrice di blocco

- Per l'implicante da espandere si costruisce una matrice (matrice di blocco):
  - che ha per indici di colonna i termini di OFFset mentre per indici di riga i letterali dell'implicante da espandere.
  - Si mette un 1 se il letterale dell'implicante compare in **forma complementata** nel cubo dell'offset

OFF-Set: a'c'+ab'd+a'd'

		a'c'	ab'd	a'd'
	a	1	0	1
ac'd'	c'	0	0	0
	d'	0	1	0

- L'espansione è risolta attraverso la soluzione di un problema di copertura: minimo numero di righe che copre la matrice

Sintesi reti combinatorie a 2 livelli: Expand – Matrice di blocco

- Osservazione: un 1 indica che se si rimuove il letterale della riga si entra in conflitto con la colonna
- Esempio

OFF-Set: a'c'+ab'd+a'd'

		a'c'	~~ab'd~~	a'd'
	a	1	0	1
ac'd'	c'	0	0	0
	d'	0	1	0

Nota: a e a' si annichiliscono
Se a non fosse parte dell'espansione il prodotto (intersezione) con OFFset non sarebbe 0. **a deve essere nella espansione**.

Rimozione di a: c'd' * (a'c' + ab'd + a'd') = a'c'd' + 0 + a'c'd'
→ questa espansione è entrata in conflitto sia con a'c' che con a'd'. Questa indicazione è riportata nella matrice di blocco. «a» serve!

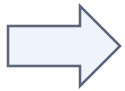
Rimozione di c': ad' * (a'c' + ab'd + a'd') = 0 + 0 + 0
→ questa espansione non è conflitto con nessun cubo dei OFF set.

Sintesi reti combinatorie a 2 livelli: Expand – Matrice di blocco

- Una copertura della matrice di blocco è una espansione valida
  - I letterali coinvolti evitano **tutte** le intersezioni con tutti i cubi di OFF set generando una espansione valida.
- Esempio

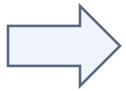
OFF-Set: a'c'+ab'd+a'd'

		a'c'	ab'd	a'd'
	a	1	0	1
ab'cd'	b'	0	0	0
	c	1	0	0
	d'	0	1	0



ad'

		a'c'	ab'd	a'd'
	a'	0	1	0
a'b'cd	b'	0	0	0
	c	1	0	0
	d	0	0	1



a'cd

- Le espansioni valide sono:
  - ac'd' → ad'
  - a'b'cd → a'cd
  - bcd → bcd
  - ab'cd' → ad'

Sintesi reti combinatorie a 2 livelli: Expand – ordinamento

	a	b	c	d		Peso	
ac'd'	01	11	10	10	⇒	3+3+2+1+2=11	(2)
a'b'cd	10	10	01	01	⇒	2+3+3+2 =10	(1)
bcd	11	01	01	01	⇒	2+3+2+3+2=12	(4)
ab'cd'	01	10	01	10	⇒	3+3+3+2 =11	(3)
							Ordine

conteggio per colonna: 23 32 13 22

Ogni peso è calcolato come prodotto interno del vettore conteggio per colonna con il vettore relativo all'implicante, espresso in notazione positional-cube.

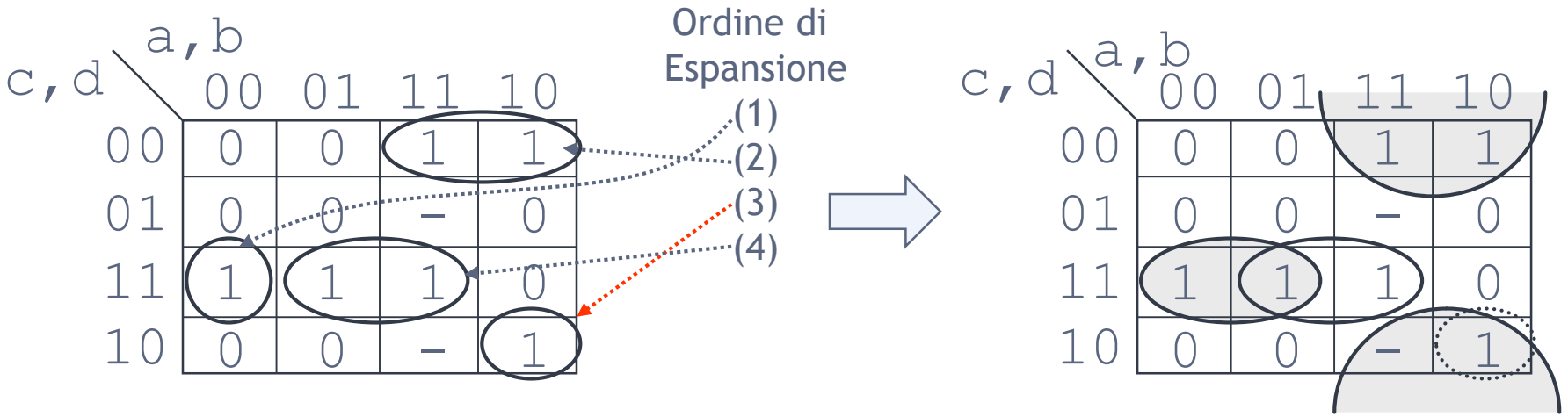
Es: $|01\ 11\ 10\ 10| * |23\ 32\ 13\ 22|^T = 11$

peso minore ⇨ espansione più probabile

- Più letterali non condivisi, dimensione più piccola
  - un cubo piccolo -più letterali- ha più probabilità di essere espanso poiché richiede pochi 1 e - adiacenti

Sintesi reti combinatorie a 2 livelli: Expand - ordinamento

- Esempio:



Copertura iniziale:

on-set: {ac'd', a'b'cd, bcd, ab'cd'}

dc-set: {abc'd, abcd'}

Copertura finale:

on-set: {ad', a'cd, bcd}

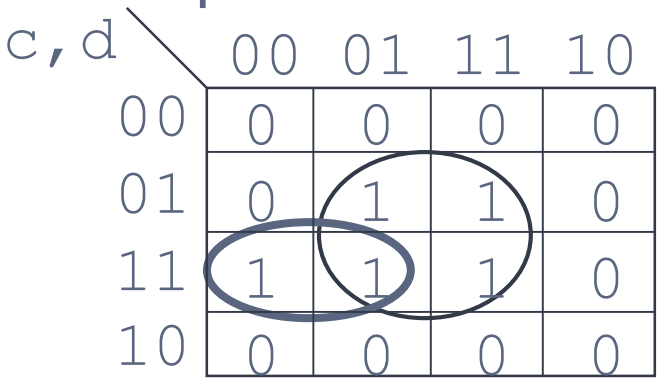
dc-set: {abc'd, abcd'}

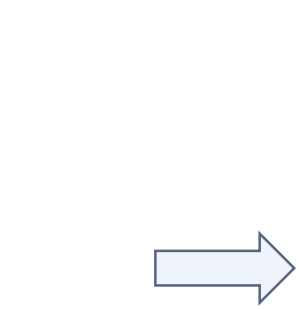
Sintesi reti combinatorie a 2 livelli: Reduce

- Reduce
  - Trasforma la copertura in un'altra non prima della stessa cardinalità.
    - Osservazione: questa trasformazione consente di uscire da minimi locali
  - Gli implicanti sono manipolati uno alla volta.
    - questa operazione mira a ridurre gli (alcuni) implicanti di dimensione per una nuova fase di espansione
    - la trasformazione di un implicante è attuata riducendo il sottocubo ad esso associato in una o più direzioni.
  - Una riduzione è ammissibile se e solo se l'implicante ridotto forma con i rimanenti una copertura per la funzione senza modificarne la cardinalità.
    - Quindi, da un implicante primo ad un implicante non primo.
  - La riduzione dipende dall'ordine di scelta

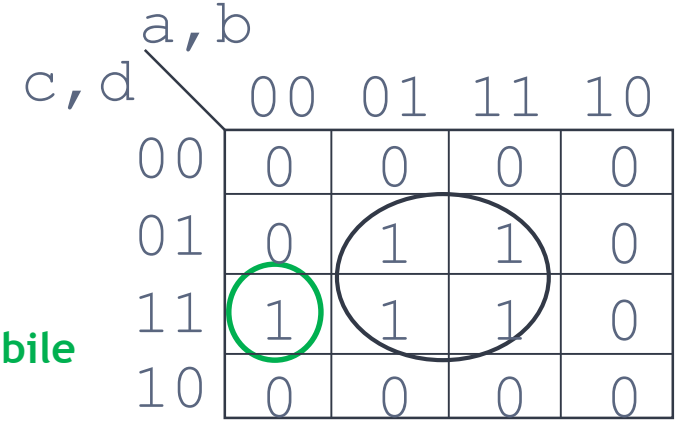
Sintesi reti combinatorie a 2 livelli: Metodi euristici: Reduce

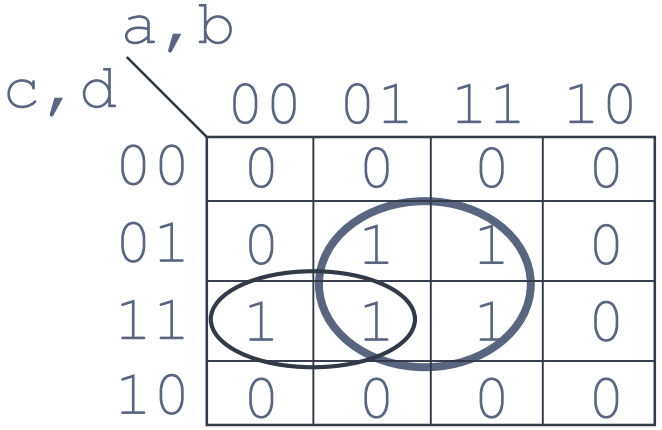
- Esempio:





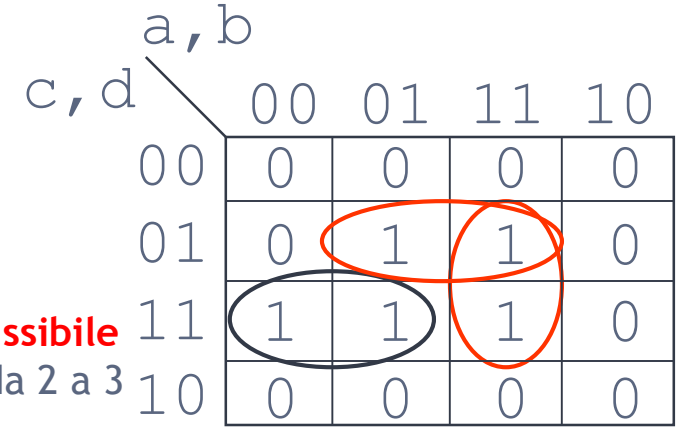
Riduzione ammissibile







Riduzione non ammissibile
La cardinalità passa da 2 a 3



A-B => bd*(a'cd)' = bd*(a+c'+d') = abd + bc'd

Sintesi reti combinatorie a 2 livelli: Reduce

- Per ridurre la probabilità di applicare la Reduce a implicanti che non possono essere ridotti oppure che portano ad una riduzione non ammissibile si applica una euristica.
- La regola euristica di scelta:
  - Il primo implicante da ridurre è quello con peso maggiore (peso calcolato come in Expand).
    - Più letterali condivisi, dimensione più ampia (quindi più facile da ridurre)
- Esempio:

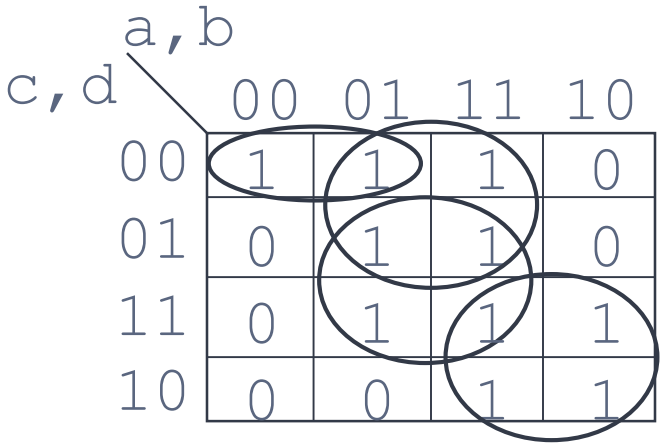
Implicanti	a	b	c	d			Ordine
ad'	01	11	11	10	⇒	3+2+3+1+3+1=13	(1)
bcd	11	01	01	01	⇒	1+3+3+3+2 =12	(3)
acd	01	11	01	01	⇒	3+2+3+3+2 =13	(2)
conteggio per colonna:	13	23	13	12			

Sintesi reti combinatorie a 2 livelli: Reduce

- Reduce parte dalla funzione di partenza e identifica l'ordine di riduzione degli implicanti primi
- Detto α l'implicante da ridurre, si determina un G costituita da tutti gli implicanti di F escludendo α.
- La riduzione è α - G ciaè α*G'
  - La riduzione è valida se non aumenta il numero degli implicanti
- La nuova funzione F è G in OR con il risultato della riduzione. Questa è la nuova funzione di partenza
  - Giustifica il fatto che il risultato dipende dall'ordine della riduzione

Sintesi reti combinatorie a 2 livelli: Reduce

- Esempio



F = a'c'd' + bc'+bd+ac

Implicanti	a b c d	ordine
a'c'd'	10 11 10 10	⇒ 15 (4)
bc'	11 01 10 11	⇒ 19 (1)
bd	11 01 11 01	⇒ 18 (2)
ac	01 11 01 11	⇒ 17 (3)
conteggio per colonna:	33 24 32 33	

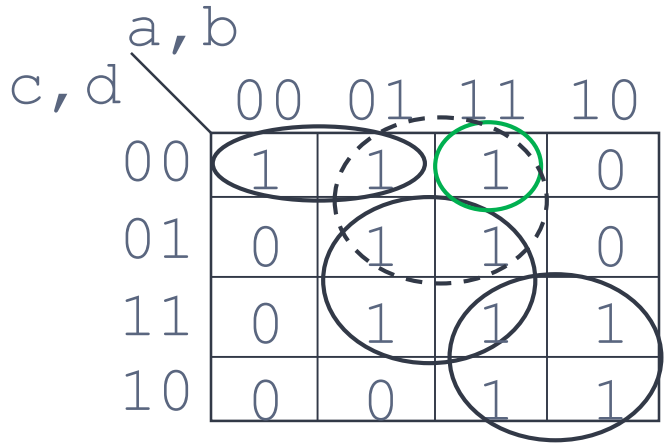
Sintesi reti combinatorie a 2 livelli: Reduce

- Esempio (cont.)
  - Riduzione di bc'

G = F-bc' = {a'c'd' , bc' , bd , ac} – {bc'} = {a'c'd' , bd , ac}

bc' – G = bc'(a'c'd' + bd + ac)' = bc'(a+c+d)(b'+d') (a'+c') = abc'd' ✓

Nuova F = a'c'd' + abc'd'+bd+ac



F = a'c'd' + abc'd'+bd+ac

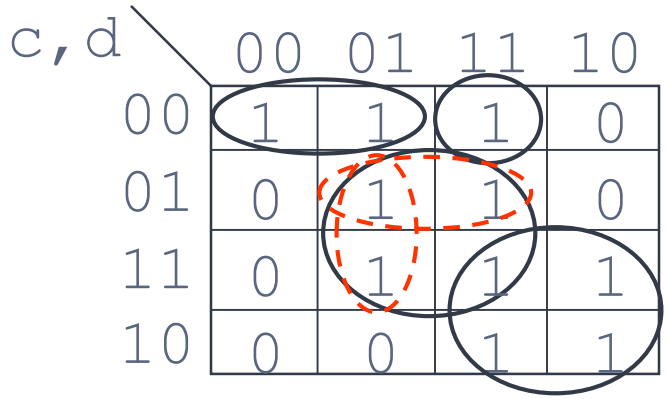
Sintesi reti combinatorie a 2 livelli: Reduce

- Esempio (cont.)
  - Riduzione di bd

G = F-bd = {a'c'd' , abc'd' , bd , ac} – {bd} = {a'c'd' , abc'd' , ac}

bd – G = bd(a'c'd' + abc'd' + ac)' = bd(a+c+d)(a'+b'+c+d) (a'+c') = a'bd+bc'd ×

Nuova F = a'c'd' + abc'd'+bd+ac



F = a'c'd' + abc'd'+bd+ac

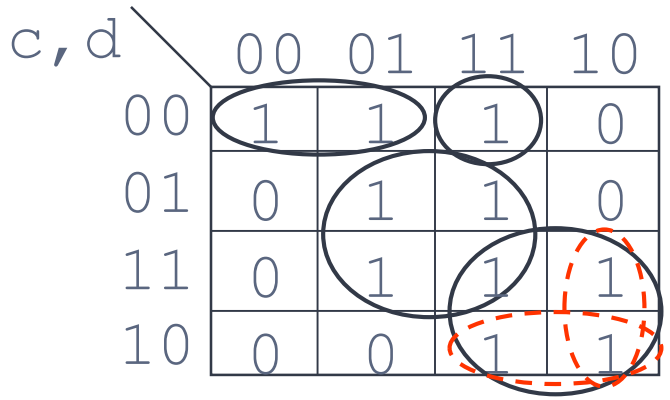
Sintesi reti combinatorie a 2 livelli: Reduce

- Esempio (cont.)
  - Riduzione di ac

G = F-ac = {a'c'd' , abc'd' , bd , ac} – {ac} = {a'c'd' , abc'd' , bd}

ac – G = ac(a'c'd' + abc'd' + bd)' = ac(a+c+d)(a'+b'+c+d) (b'+d') = ab'c+acd' ✗

Nuova F = a'c'd' + abc'd'+bd+ac



F = a'c'd' + abc'd'+bd+ac

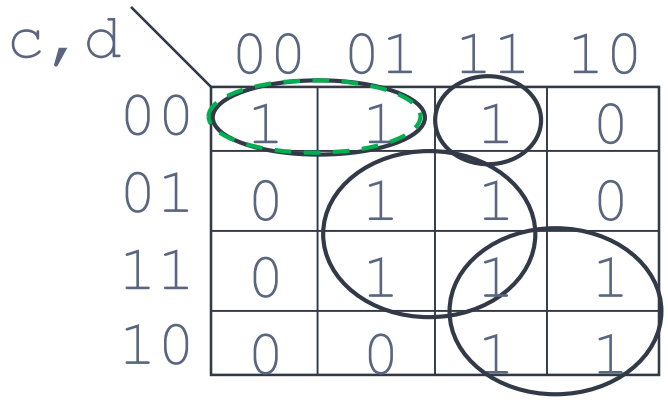
Sintesi reti combinatorie a 2 livelli: Reduce

- Esempio (cont.)
  - Riduzione di a'c'd'

G = F-ac = {a'c'd' , abc'd' , bd , ac} – {a'c'd'} = {abc'd' , bd, ac}

a'c'd' – G = a'c'd'(abc'd' + bd + ac)' = a'c'd'(a+c+d)(a'+b'+c+d) (b'+d') = a'c'd' ✓

Non è ridotto - nuova F = a'c'd' + abc'd'+bd+ac



F = a'c'd' + abc'd'+bd+ac

Sintesi reti combinatorie a 2 livelli: Irredundant

- Irredundant
  - Rende la copertura non ridondante.
  - la copertura è divisa in tre insiemi
    - relativamente essenziali (R_E): copre mintermini che non sono coperti da altri implicanti primi della copertura
    - totalmente ridondanti (R_T): tutti gli implicanti in R_T sono coperti da R_E
    - parzialmente ridondanti (R_P): implicanti che non sono ne in R_T ne in R_E
  - Lo scopo di Irredundant è determinare quali tra gli implicanti primi che appartengono all'insieme R_p può essere rimosso dalla copertura.

Sintesi reti combinatorie a 2 livelli: Irredundant

- Per identificare gli implicanti parzialmente ridondanti si utilizza l'identificazione di una tautologia. In particolare, un implicante α è parzialmente ridondante se e solo se il cofattore rispetto ad alfa di $S=R_E \cup R_P \cup DC - \alpha$ è una tautologia
  - α è coperto da altri implicanti in S
- Esempio: f=ab + a'c+ac'+bc : {ab, a'c, ac', bc}
  - α = ab; S= f- α = a'c+ac'+bc; S_α = c'+c = 1 → ab è parzialmente ridondante
  - α = a'c; S= f- α = f=ab + ac'+bc; S_α = b → a'c non è parzialmente ridonante
- Si analizzano gli implicanti, **uno alla volta**, e si escludono gli implicanti identificati come ridondanti (algoritmo greedy)

Sintesi reti combinatorie a 2 livelli: Irredundant

- Cofattore di F rispetto a α
  - Ricordiamo: il cofattore di F rispetto ad un implicante α è l'insieme delle configurazioni delle variabili non incluse in α tali per cui α implica F
    - Le configurazioni che rendono F_α è vera permettono a α di implicare F ($\alpha \rightarrow F$)
  - In particolare, se α è un implicante della funzione F, il cofattore F_α è una tautologia (Esempio: F=abc+ad ; F_{abc}=1*1*1+1*d = 1)
  - Se $\alpha \subseteq F$, quindi anche una qualunque riduzione di un cubo copertura di F, $\alpha \rightarrow F$ e, di conseguenza F_α è una tautologia

c,d \ a,b	00	01	11	10
00	0	0	0	0
01	0	1	1	0
11	0	1	1	1
10	0	0	0	0

F = bd + acd

$F_{abcd} = 1*1 + 1*1*1 = 1$

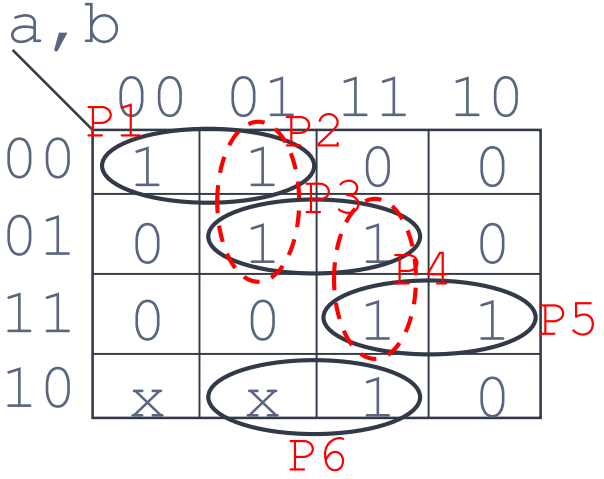
$F_{bc'd} = 1*1 + a*0*1 = 1$

Sintesi reti combinatorie a 2 livelli: Irredundant

- Esempio: F=a'c'd'+a'bc'+bc'd+abd+acd+bcd' - DC=a'cd'
  - Analisi di a'c'd':
    - **G**=F-a'c'd' = a'bc'+bc'd+abd+acd+bcd' +a'cd'
    - $G_{a'c'd'}$ = b non è ridondante
  - Analisi di a'bc':
    - **G**=F-a'bd' = a'c'd'+ bc'd+abd+acd+bcd' +a'cd'
    - $G_{a'bc'}$ = d'+ d+ … = 1 (tautologia) ridondante. Eliminato dalla copertura
  - Nuovo: F=a'c'd'+ bc'd+abd+acd+bcd' - DC=a'cd'
  - Analisi di bc'd:
    - **G**=F-bc'd = a'c'd' +abd+acd+bcd' +a'cd'
    - $G_{a'bc'}$ = d' non è ridondante

Sintesi reti combinatorie a 2 livelli: Irredundant

- (cont) Nuovo: F=a'c'd'+ bc'd+abd+acd+bcd' - DC=a'cd'
- Analisi di abd:
  - **G**=F-abd= a'c'd' +bc'd+acd+bcd' +a'cd'
  - G_{abd} = c'+c+… = 1 (tautologia) ridondante. Eliminato dalla copertura
- Nuovo: F=a'c'd'+ bc'd+acd+bcd' - DC=a'cd'
- Analisi di acd:
  - **G**=F-acd = a'c'd'+ bc'd+bcd' +a'cd'
  - G_{acd} = 0 non è ridondante
- Analisi di bcd':
  - **G**=F-bcd' = a'c'd'+ bc'd+acd +a'cd'
  - $G_{bcd'}$ = a' non è ridondante

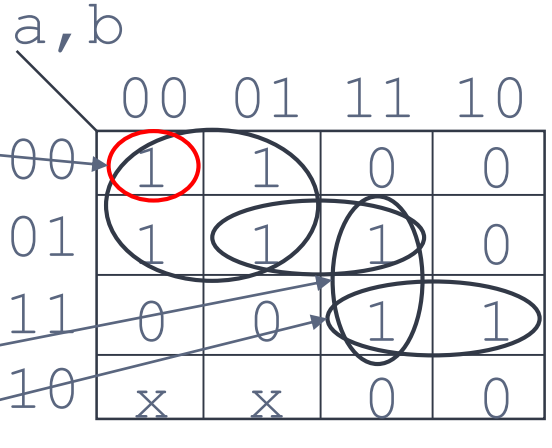


- Nota: il risultato dipende dall'ordine. Partire da P3 avrebbe prodotto un risultato diverso (5 implicanti al posto di 4)

Sintesi reti combinatorie a 2 livelli: Irredundant

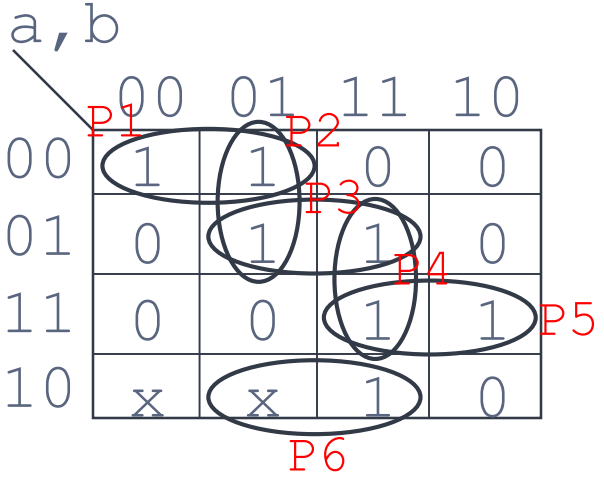
- Un potenziale criterio di scelta è dato dal criterio di costo proposto dalla notazione posizionale: il cubo con valore più basso è il più interessante e da analizzare per primo
  - Es: completamente coperto da un altro cubo

Implicanti	a b c d		ordine
a'b'c'd'	10 10 10 10	⇒ 13	(1)
a'c'	10 11 10 11	⇒ 20	(5)
bc'd	11 01 10 01	⇒ 17	(4)
abd	01 01 11 10	⇒ 16	(3)
acd	01 11 01 01	⇒ 15	(2)
conteggio per colonna:	33 34 42 33		



Sintesi reti combinatorie a 2 livelli: Irredundant

- Identificazione di una copertura non ridondante prima in modo esatto
  - Nota: non sono tutti gli implicanti primi possibile ma solo quelli della copertura identificata. Tabella più piccola di QM



	0	4	5	11	13	14	15
P1	x	x					
P2		x	x				
P3			x		x		
P4					x		x
P5				x			x
P6						x	

$$G(P) = P1(P1+P2)(P2+P3)P5(P3+P4)P6(P4+P5) =$$
$$= P1P5P6(P2+P3)(P3+P4) = P1P5P6(P2P4+P3)$$

Copertura non ridondante: P1 P5 P6 P3

Sintesi reti combinatorie a 2 livelli: Metodi euristici: ESPRESSO

- (1982) ESPRESSO II è basato sulla applicazione di iterate espansioni e riduzioni. Il risultato prodotto da ESPRESSO è una copertura non ridondante spesso di minima cardinalità.
- I passi seguiti da ESPRESSO II sono:
  1. COMPLEMENT: Calcola l'OFFset dato ONset U DCset.
  2. EXPAND: Espande a primi gli implicanti e rimuove quelli coperti.
  3. ESSENTIAL PRIMES: Estrae gli implicanti essenziali primi e li unisce al DC-set.
  4. **REDUCE: Riduce ogni implicante a un implicante essenziale minimo.**
  5. **EXPAND: Espande a primi gli implicanti e rimuove quelli coperti.**
  6. **IRREDUNDANT COVER: Trova la copertura minimale non ridondante. Se la soluzione migliora vai al passo 4.**
  7. LASTGASP: applica per un'ultima volta REDUCE, EXPAND e IRREDUNDANT COVER usando una differente strategia.
  8. COST: Se questa operazione ha successo vai al passo 4.
  9. MAKESPARSE: Adatta la soluzione ad una PLA.

Sintesi reti combinatorie a 2 livelli: Metodi euristici: ESPRESSO

- Espresso(I)

  (F,D,R) ⇦ DECODE(I) //F ONset, D DCset, R OFFset

  F ⇦ EXPAND(F,R)

  F ⇦ IRREDUNDANT(F,D)

  E ⇦ ESSENTIAL_PRIMES(F,D)

  F ⇦ F-E ; D ⇦ D+E //i passi successivi operano solo sugli implicanti non essenziali

  do{

  do{

  F ⇦ REDUCE(F,D)

  F ⇦ EXPAND(F,R)

  F ⇦ IRREDUNDANT(F,D)

  } fino a che i termini in F non sono più riducibili

  LAST_GASP()

  }

  F ⇦ F+E ; D ⇦ D-E

Sintesi reti combinatorie a 2 livelli: Metodi euristici: ESPRESSO

- Espresso(on_set,dc_set)

```
off_set=Complement(on_set U dc_set)
on_set=Expand(on_set, off_set)  /*copertura prima ridondante*/
on_set=Irredundant(on_set, dc_set)
essential_set=Essentials(on_set, dc_set)
on_set=on_set - essential_set  /* toglie 1 dall'on_set  */
dc_set=dc_set U  essential_set /* e li aggiunge al dc_set */
ripeti
        f2=Cost(on_set)
        ripeti
                f1=|on_set|
                on_set=Reduce(on_set,dc_set)
                on_set=Expand(on_set, off_set)
                on_set=Irredundant(on_set,dc_set)
        fino a che (|on_set|< f1)
        on_set=Last_gasp(on_set,dc_set,off_set)
fino a che (Cost(on_set) < f2)
on_set=on_set U essential_set
dc_set=dc_set - essential_set
on_set=Make_sparse(on_set,dc_set,off_set)
```

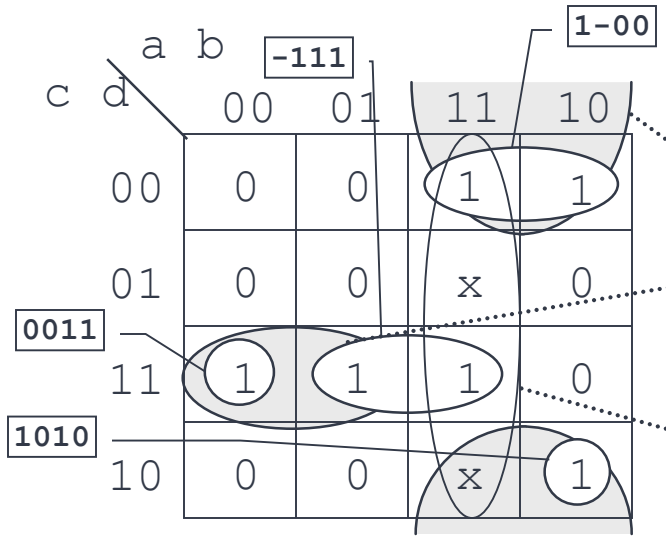
Sintesi reti combinatorie a 2 livelli: ESPRESSO

- Comando:
  - espresso [parametri] [file]
- Funzione:
  - minimizzazione di funzioni logiche a due livelli.
- Parametri:
  - -d: debugging
  - -e[opzioni]: seleziona le opzioni di espresso:
    - fast, ness, nirr, nunwrap, onset, pos, strong, eat, eatdots, kiss, random
  - -o[tipo]: seleziona il formato di uscita:
    - f, fd, fr, fdr, pleasure, eqntott, kiss, cons
  - -s: fornisce un breve sommario relativo all'esecuzione;
  - -t: fornisce un ampio sommario relativo all'esecuzione;
  - -x: non visualizza la soluzione;

Sintesi reti combinatorie a 2 livelli: ESPRESSO

- **Parametri (continua):**
  - -v[tipo]: messaggi di dettaglio (-v '' per un accurato dettaglio)
  - -D[comando]: esegue il sotto-comando:
    - ESPRESSO, many, exact, qm, single_output, so, so_both, simplify, echo, opo, opoall, pair, pairall, check, stats, verify, PLAverify, equiv, map, mapdc, fsm, contain, d1merge, d1merge_in, disjoint, dsharp, intersect, minterms, primes, separate, sharp, union, xor, essen, expand, gasp, irred, make_sparse, reduce, taut, super_gasp, lexsort, test
  - -Sn: seleziona la strategia per il sotto comando (solo quelli riportati):
    - opo: bit2=esatto, bit1=ripetuto bit0=salta sparse
    - opoall: 0=minimizza, 1=esatto
    - pair: 0=algebrico, 1=strongd, 2=espresso, 3=esatto
    - pairall: 0=minimizza, 1=esatto, 2=opo
    - so_espresso: 0=minimize, 1=exact
    - so_both: 0=minimize, 1=exact

Sintesi reti combinatorie a 2 livelli: ESPRESSO




```
.i 4
.o 1
.type fd
1-00 1
0011 1
-111 1
1010 1
1101 -
1110 -
```

```
ipeca4>espresso -v " ex3.pla
EXPAND: 0011 1 (covered 0)
EXPAND: 1-00 1 (covered 1)
EXPAND: -111 1 (covered 0)
# IRRED: F=3 E=3 R=0 Rt=0 Rp=0 Rc=0 Final=3 Bound=0
ESSENTIAL: 0-11 1
ESSENTIAL: 1--0 1
REDUCE: -111 1 to 1111 1 0.00 sec
EXPAND: 1111 1 (covered 0)
# IRRED: F=1 E=1 R=0 Rt=0 Rp=0 Rc=0 Final=1 Bound=0
REDUCE_GASP: 11-- 1 reduced to 1111 1
# IRRED: F=3 E=3 R=0 Rt=0 Rp=0 Rc=0 Final=3 Bound=0
.i 4
.o 1
.p 3
11-- 1
0-11 1
1--0 1
.e
ipeca4>
```

Cubi	Notazione positional-cube	Pesi	Ordine
1-00	1 0 1 1 0 1 0 1	11	**2**
0011	0 1 0 1 1 0 1 0	10	**1**
-111	1 1 1 0 1 0 1 0	12	**4**
1010	1 0 0 1 1 0 0 1	11	**3**
	2 3 3 2 1 3 2 2		

Sintesi reti combinatorie a 2 livelli: ESPRESSO

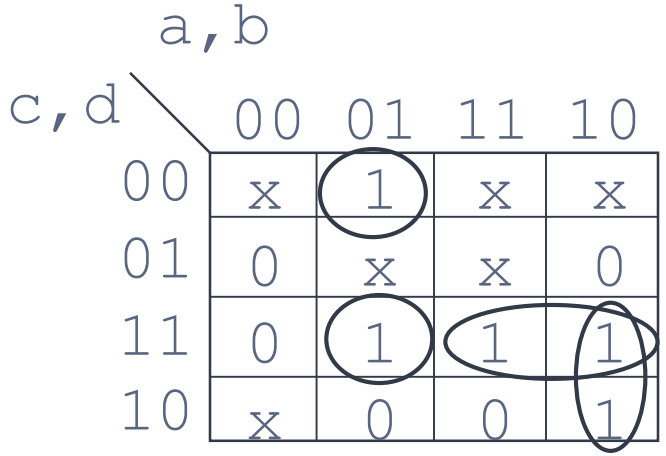
- Esempio:

```
.i 4
.o 3
.type fr
00-1 1-1
01-- 001
1000 0--
1001 --1
1011 -1-
1100 111
1101 000
1110 -10
1111 01-
```

```
EXPAND: 1100 100 (covered 2)
EXPAND: 1011 010 (covered 3)
EXPAND: 1111 010 (covered 0)
EXPAND: 1100 001 (covered 0)
EXPAND: 01-- 001 (covered 0)
# IRRED: F=5 E=5 R=0 Rt=0 Rp=0 Rc=0 Final=5
Bound=0
ESSENTIAL: 0--- 001
REDUCE: -0-1 111 to -0-1 101 0.00 sec
REDUCE: 1-1- 010 to 1-11 010 0.00 sec
REDUCE: 1-00 011 to 1100 001 0.00 sec
EXPAND: 1100 001 (covered 0)
EXPAND: 1-11 010 (covered 0)
EXPAND: -0-1 101 (covered 0)
# IRRED: F=4 E=4 R=0 Rt=0 Rp=0 Rc=0 Final=4
Bound=0
REDUCE_GASP: 1-00 011 reduced to 1100 001
REDUCE_GASP: 1-1- 010 reduced to 1111 010
REDUCE_GASP: 11-0 110 reduced to 1100 100
REDUCE_GASP: -0-1 111 reduced to -0-1 101
EXPAND: 1100 111 (covered 0)
# IRRED: F=5 E=2 R=3 Rt=0 Rp=3 Rc=1 Final=3
Bound=0
REDUCE: -0-1 111 to -0-1 101 0.01 sec
EXPAND: -0-1 101 (covered 0)
# IRRED: F=3 E=3 R=0 Rt=0 Rp=0 Rc=0 Final=3
Bound=0
… (continua)
```

```
… (continua)
# IRRED: F=3 E=3 R=0 Rt=0 Rp=0 Rc=0 Final=3 Bound=0
REDUCE_GASP: 1-1- 010 reduced to 111- 010
REDUCE_GASP: 1100 111 reduced to 1100 111
REDUCE_GASP: -0-1 111 reduced to -0-1 101
# IRRED: F=2 E=2 R=0 Rt=0 Rp=0 Rc=0 Final=2 Bound=0
# IRRED: F=3 E=2 R=1 Rt=1 Rp=0 Rc=0 Final=2 Bound=0
# IRRED: F=3 E=3 R=0 Rt=0 Rp=0 Rc=0 Final=3 Bound=0
EXPAND: -0-1 101 (covered 0)
.i 4
.o 3
.p 4
1100 111
1-1- 010
-0-1 101
0--- 001
.e
```

Esempio



F :{a'bc'd', a'bcd, acd, ab'c}

D :{b'c'd', abc', bc'd, a'b'd'}

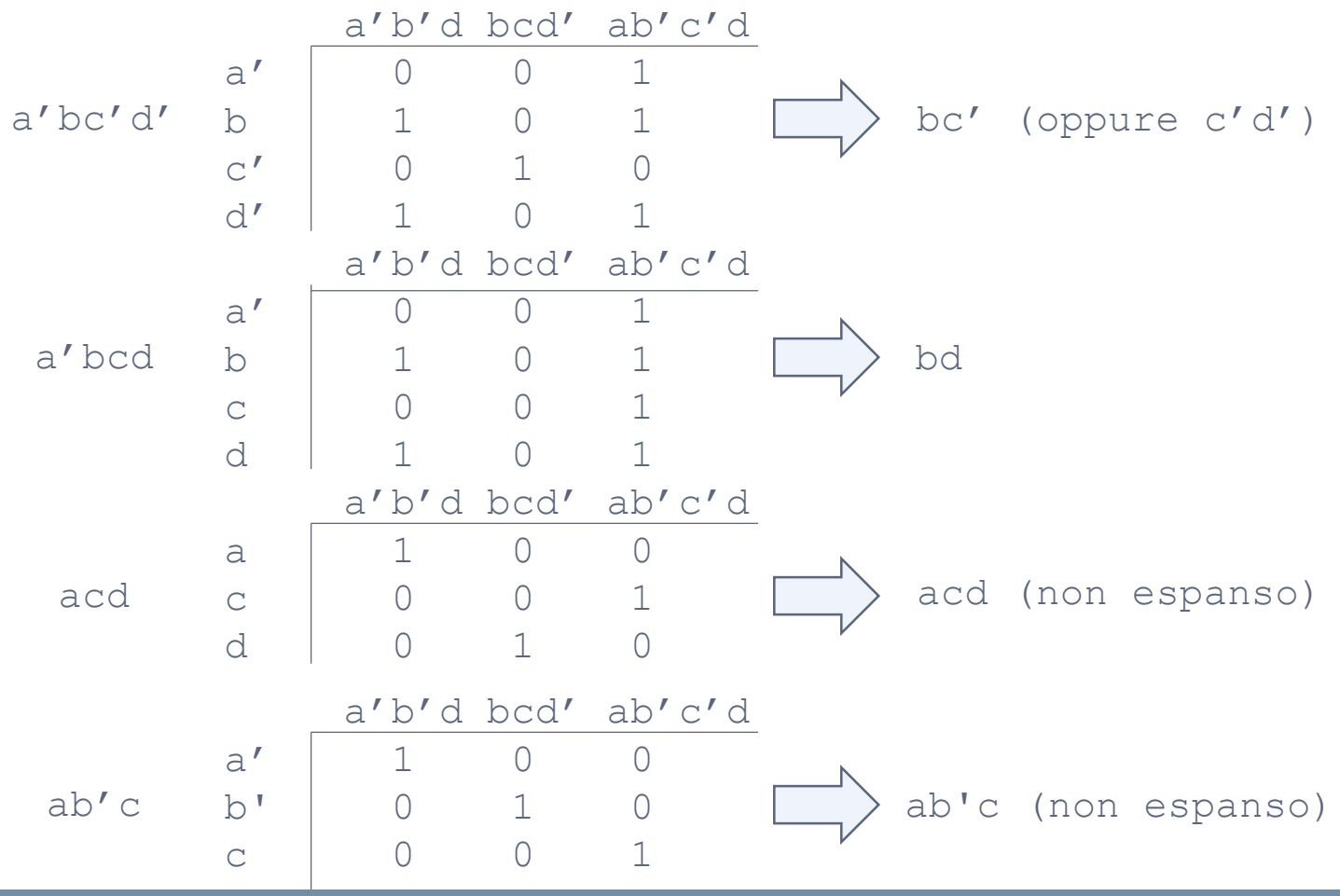
R :{a'b'd, bcd', ab'c'd}

- EXPAND

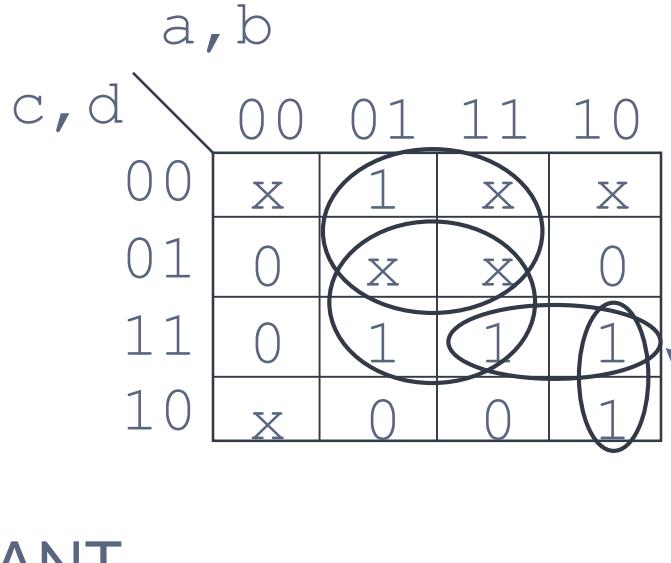
Implicanti	a b c d		ordine
a'bc'd'	10 01 10 10	⇒ 8	(1)
a'bcd	10 01 01 01	⇒ 11	(2)
acd	01 11 01 01	⇒ 13	(4)
ab'c	01 10 01 11	⇒ 12	(3)
conteggio per colonna:	22 23 13 23		

Esempio

- (cont.)



Esempio



F :{bc', bd, acd, ab'c}

D :{b'c'd', abc', bc'd, a'b'd'}

R :{a'b'd, bcd', ab'c'd}

- IRREDUNDANT
  - Analisi per bc'

    F ∪D= bc' + bd + acd + ab'c + b'c'd' + abc' + bc'd + a'b'd'

    S = F ∪D – bc' = bd + acd + ab'c + b'c'd' + abc' + bc'd + a'b'd'

    $S_{bc'}$ = d+a+d = d+a Non ridondante

Esempio

- Cont.
  - Analisi per bd

    F ∪D= bc’ + bd + acd + ab’c + b’c’d’ + abc’ + bc’d + a’b’d’

    S = F ∪D – bd = bc’ + acd + ab’c + b’c’d’ + abc’ + bc’d + a’b’d’

    S_{bd} = c’+ac+ac’+c’ = c’+a Non ridondante
  - Analisi per acd

    F ∪D= bc’ + bd + acd + ab’c + b’c’d’ + abc’ + bc’d + a’b’d’

    S = F ∪D – acd = bc’+ bd + ab’c + b’c’d’ + abc’ + bc’d + a’b’d’

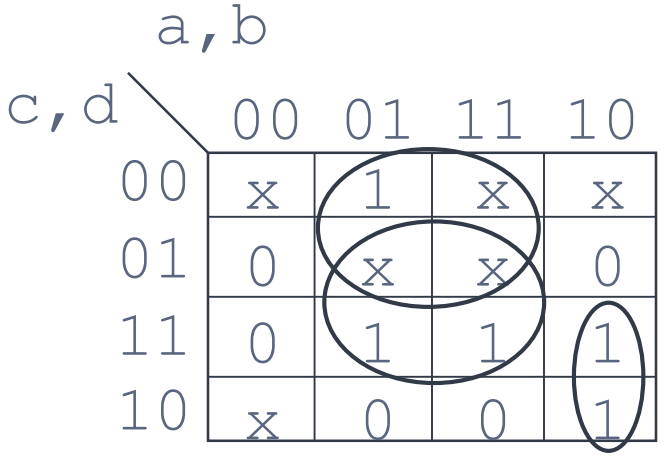
    S_{acd} = b + b’ = 1 **ridondante**
  - Analisi per ab’c

    F ∪D= bc’ + bd + ~~acd +~~ ab’c + b’c’d’ + abc’ + bc’d + a’b’d’

    S = F ∪D – ab’c = bc’ + bd + b’c’d’ + abc’ + bc’d + a’b’d’

    $S_{ab'c}$ = d Non ridondante

Esempio



F :{bc', bd, ab'c}

D :{b'c'd', abc', bc'd, a'b'd'}

R :{a'b'd, bcd', ab'c'd}

- REDUCE

Implicanti	a b	c d		ordine
bc'	11 01	10 11	⇒ 14	(1)
bd	11 01	11 01	⇒ 14	(2)
ab'c	01 10	01 11	⇒ 11	(3)
conteggio per colonna:	23 12	22 23		

Esempio

- Cont.
  - Riduzione di bc' (1)

    G= F-bc' = {ab'c , bc' , bd} –{bc'} = {ab'c , bd}

    bc' - G= bc' (ab'c + bd)' = bc' (a'+b+c')(b'+d') = (a'bc'+c'b)(b'+d')

    = a'bc'd' + bc'd' = bc'd' (**ridotto**)

    F_{new} = ab'c + bc'd' + bd

  - Riduzione di bd (2)

    G= F-bd = {ab'c , bc'd' , bd} –{bd} = {ab'c , bc'd'}

    bd - G= bd (ab'c + bc'd')' = bd (a'+b+c')(b'+c+d) = bd (a'+b+c')

    = bd (non ridotto)

    F_{new} = ab'c + bc'd' + bd

Esempio

- Cont.
  - Riduzione di ab'c (3)

    G= F-bd = {ab'c , bc'd' , bd} –{ab'c} = {bc'd' , bd}

    ab'c - G= ab'c (bc'd' + bd)' = ab'c (b'+c+d)(b'd') =

    = ab'c (non ridotto)

    F_{new} = ab'c + bc'd' + bd

c,d \ a,b	00	01	11	10
00	x	1	x	x
01	0	x	x	0
11	0	1	1	1
10	x	0	0	1

F :{bc'd', bd, ab'c}

D :{b'c'd', abc', bc'd, a'b'd'}

R :{a'b'd, bcd', ab'c'd}

Esempio

- EXPAND F :{bc'd', bd, ab'c}

Implicanti	a b	c d		ordine
bc'd'	11 01	10 10	⇒ 11	(2)
bd	11 01	11 01	⇒ 14	(3)
ab'c	01 10	01 11	⇒ 10	(1)

conteggio per colonna: 23 12 22 22

		a'b'd	bcd'	ab'c'd
	a	1	0	0
ab'c'	b'	0	1	0
	c	0	0	1

⇒ ab'c (nessuna espansione)

		a'b'd	bcd'	ab'c'd
	b	1	0	1
bc'd'	c'	0	1	0
	d'	1	0	1

⇒ bc' (oppure c'd')

Soluzione di partenza - Fine

Home Work

- Data la seguente funzione:
  - ON:{a'b'c'd', c'd, ab'c, b'cd', a'b'd'} – DC:{a'bc'd' , abcd}
  - Calcolare OFF (algebricamente)
    - $F' = x (f_x)' + x' (f_{x'})'$
  - Applicare EXPAND:
    - Ordine dell'espansione
    - Matrice di Blocco (Blocking Matrix) e Espansione
  - Applicare IRREDUNDANT:
    - Costruire la matrice di copertura
    - Selezionare gli implicanti che costituiscono una buona copertura
  - Applicare REDUCE

Esercizio

- Data la seguente funzione:
  - ON:{a'c', c'd, ac, cd'} – DC: ∅
  - Calcolare OFF (algebricamente)
    - $F' = x (f_x)' + x' (f_{x'})'$
  - Applicare REDUCE:
    - Ordine di riduzione
    - Applicare la riduzione
  - Applicare IRREDUNDANT:
    - Costruire la matrice di copertura
    - Selezionare gli implicanti che costituiscono una buona copertura
  - Applicare EXPAND:
    - Ordine dell'espansione
    - Matrice di Blocco (Blocking Matrix) e Espansione